

NETWORK TRAFFIC ANALYSIS AND SUSPICIOUS ACTIVITY DETECTION USING WIRESHARK



Cybersecurity Internship Project

Submitted by:

Dibyasmitha Acharya

**B.Tech in Computer Science and Engineering
Specialization in Cybersecurity**

Tools Used:

Wireshark 4.6.8, Nmap 7.991, Npcap 1.88

Operating Environment - Windows 11

Submitted for - Cybersecurity Internship

Organization - CodeC Technologies

Academic Year - 2026

TABLE OF CONTENTS

1. ABSTRACT	3
2. OBJECTIVES	4
3. TOOLS AND TECHNOLOGIES	5
4. METHODOLOGY	6
4.1 Normal Traffic Capture	6
4.2 DNS Analysis	6
4.3 TCP and TLS Analysis	7
4.3.1 TCP Three-Way Handshake	7
4.3.2 TLS Client Hello	8
4.4 Controlled SYN Port Scan	8
4.4.1 SYN Scan Detection	9
4.4.2 RST/ACK Responses	9
5. NETWORK TRAFFIC ANALYSIS	10
5.1 DNS Query and Response Analysis	10
5.2 TCP Three-Way Handshake Analysis	10
5.3 TLS Client Hello Analysis	10
6. SUSPICIOUS ACTIVITY DETECTION	11
6.1 SYN Port Scan Analysis	11
6.2 RST/ACK Response Analysis	11
7. PROTOCOL HIERARCHY ANALYSIS	12
7.1 Protocol Distribution	12
8. KEY FINDINGS	13
9. CONCLUSION	14
10. REFERENCES	15

1. ABSTRACT

Traffic analysis is an essential procedure of cybersecurity that helps understand the connections between devices and discover network protocols and suspicious behavior.

This project provides an example of traffic analysis conducted using Wireshark. To have the ground for future analysis, the first step is to make traffic capture in a normal situation. This traffic will be then analyzed considering DNS requests and responses, TCP connections, TLS Client Hello packets and protocol distribution.

The detection of suspicious traffic in a controlled environment was conducted through SYN port scanning of the local host utilizing Nmap, as well as the capture of TCP packets with SYN flag, directed at various ports, with the respective RST/ACK responses with the use of Wireshark.

This project demonstrates how traffic analysis can be used to detect normal and reconnaissance traffic behavior.

2. OBJECTIVES

1. To understand the fundamentals of network packet analysis.
2. To capture and analyze normal network traffic using Wireshark.
3. To analyze DNS queries and responses within captured network traffic.
4. To understand the TCP three-way handshake.
5. To examine information contained in TLS Client Hello packets.
6. To analyze the distribution of protocols within captured network traffic.
7. To simulate a controlled TCP SYN port scan using Nmap.
8. To identify characteristics of port-scanning activity using Wireshark.
9. To understand how network traffic analysis can assist in detecting suspicious activity.

3. TOOLS AND TECHNOLOGIES

Tool / Technology	Version	Purpose
Wireshark	4.6.8	Packet capture and network traffic analysis
Npcap	1.88	Packet capture support on Windows
Nmap	7.991	Controlled SYN port-scan simulation
Windows	11	Operating environment
TCP/IP	—	Network communication protocols analyzed
DNS	—	Domain name resolution analysis
TLS	—	Encrypted communication analysis

4. METHODOLOGY

4.1 Normal Traffic Capture

A baseline capture was performed using Wireshark to analyze normal network traffic during regular web browsing activity. In this case, the packet capture was done on the Wi-Fi network interface. Normal network traffic packets were then analyzed using the Wireshark packet analyzer, and the packets were stored in the `normal_traffic_baseline.pcapng` file.

There were different types of protocols present in the network baseline, which included TCP, UDP, DNS, TLS, QUIC, IPv4, and IPv6 traffic.

4.2 DNS Analysis

- DNS filtering was applied in Wireshark
- A DNS query from the local machine to the DNS resolver was identified.
- Examined a query for a website.
- Examined the corresponding DNS response.
- The response showed **No error** and contained multiple A-record answers.
- The response time was approximately **4.75 ms**.
- This demonstrates the normal domain-name-resolution process.

No.	Time	Source	Destination	Protocol	Length	Info
64	6.117081100	192.168.31.196	192.168.31.1	DNS	79	Standard query 0xf152 HTTPS accounts.google.com
65	6.118022300	192.168.31.196	192.168.31.1	DNS	79	Standard query 0xe104 AAAA accounts.google.com
66	6.118938600	192.168.31.196	192.168.31.1	DNS	79	Standard query 0xafdc A accounts.google.com
67	6.139948300	192.168.31.1	192.168.31.196	DNS	107	Standard query response 0xe104 AAAA accounts.google.com
68	6.139951100	192.168.31.1	192.168.31.196	DNS	129	Standard query response 0xf152 HTTPS accounts.google.com
69	6.139952100	192.168.31.1	192.168.31.196	DNS	95	Standard query response 0xafdc A accounts.google.com
102	6.475539000	192.168.31.196	192.168.31.1	DNS	75	Standard query 0x11c4 A www.youtube.com
108	6.486600300	192.168.31.1	192.168.31.196	DNS	203	Standard query response 0x11c4 A www.youtube.com A 14
110	6.505147600	192.168.31.196	192.168.31.1	DNS	75	Standard query 0xa4b8 HTTPS www.youtube.com
111	6.506110600	192.168.31.196	192.168.31.1	DNS	75	Standard query 0x5b1f AAAA www.youtube.com
112	6.507080700	192.168.31.196	192.168.31.1	DNS	75	Standard query 0x904e A www.youtube.com
113	6.511829800	192.168.31.1	192.168.31.196	DNS	203	Standard query response 0x904e A www.youtube.com A 14
114	6.514745800	192.168.31.1	192.168.31.196	DNS	90	Standard query response 0xa4b8 HTTPS www.youtube.com
115	6.517458000	192.168.31.1	192.168.31.196	DNS	299	Standard query response 0x5b1f AAAA www.youtube.com A

Frame 112: Packet, 75 bytes on wire (600 bits), 75 bytes captured	0000	fc b0 de 2c 13 9f 0c 7a 15 de 38 e3 08 00 45 00	...
Ethernet II, Src: Intel_de:38:e3 (0c:7a:15:de:38:e3), Dst: CloudNe	0010	00 3d 24 e3 00 00 80 11 55 b7 c0 a8 1f c4 c0 a8	=\$..
Internet Protocol Version 4, Src: 192.168.31.196, Dst: 192.168.31.	0020	1f 01 de f0 00 35 00 29 91 88 90 4e 01 00 00 01	5
User Datagram Protocol, Src Port: 57072, Dst Port: 53	0030	00 00 00 00 00 00 03 77 77 77 07 79 6f 75 74 75	
Domain Name System (query)	0040	62 65 03 63 6f 6d 00 00 01 00 01	be.com
Transaction ID: 0x904e			
Flags: 0x0100 Standard query			
Questions: 1			
Answer RRs: 0			
Authority RRs: 0			
Additional RRs: 0			
Queries			
[Response In: 113]			

Figure 1: DNS Query

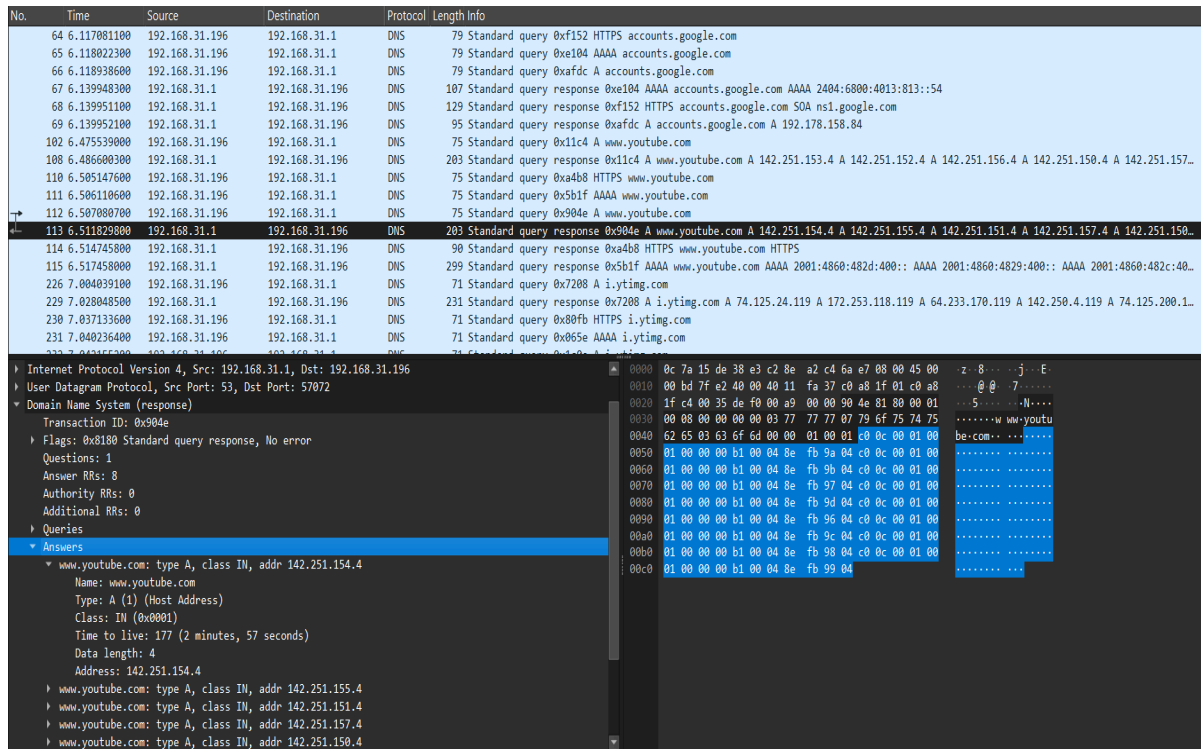


Figure 2: DNS Response

4.3 TCP and TLS Analysis

4.3.1 TCP Three-Way Handshake

- Connection was established via a TCP connection from the captured data.
- The connection was initiated with the transmission of a SYN packet.
- The destination replied to the source with a SYN/ACK packet.
- An ACK packet was transmitted by the source.
- This forms part of the normal TCP handshake process.
- The filtering criterion used was **tcp.flags.syn == 1**.

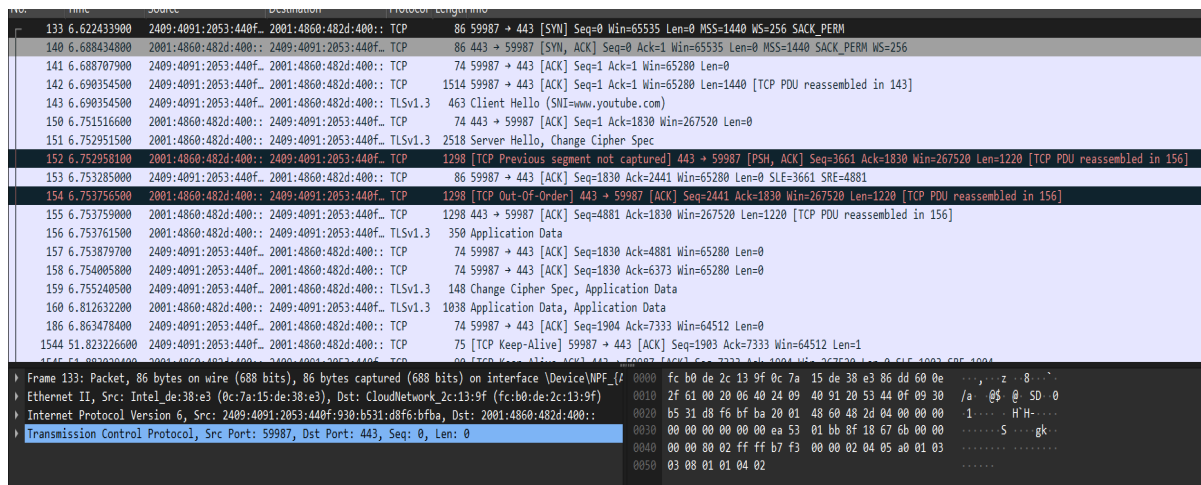


Figure 3: TCP Three-Way Handshake

4.3.2 TLS Client Hello

- A TLS 1.3 Client Hello message was detected.
- The message had an SNI (Server Name Indication) extension field.
- The Server Name indicated that the requested hostname was www.youtube.com.
- The Client Hello message is used in the TLS handshake process.
- The capture also demonstrated the use of TLS for encrypted communication.

No.	Time	Source	Destination	Protocol	Length	Info
133	6.622433900	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	86	59987 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
140	6.688434800	2001:4860:482d:400::	2409:4091:2053:440f::	TCP	86	443 → 59987 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1440 SACK_PERM WS=256
141	6.688707900	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	74	59987 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
142	6.690354500	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	1514	59987 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1440 [TCP PDU reassembled in 143]
143	6.690354500	2409:4091:2053:440f::	2001:4860:482d:400::	TLSv1.3	463	Client Hello (SNI=www.youtube.com)
150	6.751516600	2001:4860:482d:400::	2409:4091:2053:440f::	TCP	74	443 → 59987 [ACK] Seq=1 Ack=1830 Win=267520 Len=0
151	6.752951500	2001:4860:482d:400::	2409:4091:2053:440f::	TLSv1.3	2518	Server Hello, Change Cipher Spec
152	6.752958100	2001:4860:482d:400::	2409:4091:2053:440f::	TCP	1298	[TCP Previous segment not captured] 443 → 59987 [PSH, ACK] Seq=3661 Ack=1830 Win=267520 Len=1220 [TCP PDU reassembled in 156]
153	6.753755000	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	86	59987 → 443 [ACK] Seq=1830 Ack=2441 Win=65280 Len=0 SLE=3661 SRE=4881
154	6.753755000	2001:4860:482d:400::	2409:4091:2053:440f::	TCP	1298	[TCP Out-Of-Order] 443 → 59987 [ACK] Seq=4881 Ack=1830 Win=267520 Len=1220 [TCP PDU reassembled in 156]
155	6.753759000	2001:4860:482d:400::	2409:4091:2053:440f::	TCP	1298	443 → 59987 [ACK] Seq=4881 Ack=1830 Win=267520 Len=1220 [TCP PDU reassembled in 156]
156	6.753761500	2001:4860:482d:400::	2409:4091:2053:440f::	TLSv1.3	350	Application Data
157	6.753879700	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	74	59987 → 443 [ACK] Seq=1830 Ack=4881 Win=65280 Len=0
158	6.754005800	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	74	59987 → 443 [ACK] Seq=1830 Ack=6373 Win=65280 Len=0
159	6.755240500	2409:4091:2053:440f::	2001:4860:482d:400::	TLSv1.3	148	Change Cipher Spec, Application Data
160	6.812632200	2001:4860:482d:400::	2409:4091:2053:440f::	TLSv1.3	1038	Application Data, Application Data
186	6.863478400	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	74	59987 → 443 [ACK] Seq=1904 Ack=7333 Win=64512 Len=0
1544	51.823226600	2409:4091:2053:440f::	2001:4860:482d:400::	TCP	75	[TCP Keep-Alive] 59987 → 443 [ACK] Seq=1903 Ack=7333 Win=64512 Len=1

Extension: signed_certificate_timestamp (len=0)
Extension: application_layer_protocol_negotiation (len=14)
Extension: server_name (len=20) name=www.youtube.com
Type: server_name (0)
Length: 20
Server Name Indication extension
Server Name list length: 18
Server Name Type: host_name (0)
Server Name length: 15
Server Name: www.youtube.com
Extension: session_ticket (len=0)
Extension: renegotiation_info (len=1)
Extension: signature_algorithms (len=26)
Extension: status_request (len=5)
Extension: key_share (len=1263) X25519MLKEM768, x25519
Extension: supported_versions (len=7) TLS 1.3, TLS 1.2
Extension: application_settings (len=5)
Extension: encrypted_client_hello (len=282)
Extension: psk_key_exchange_modes (len=2)
Extension: ec_point_formats (len=2)
Extension: extended_master_secret (len=0)
Extension: compress_certificate (len=3)

Packet (463 bytes) Reassembled TCP (1829 bytes)

Packets: 2068 · Displayed: 19 (0.9%) · Dropped: 0 (0.0%) Profile:

Figure 4: TLS Client Hello

4.4 Controlled SYN Port Scan

- Nmap software was installed and used for scanning.
- The scanning was done on the local machine.
- The IP address of the target was **127.0.0.1**.
- The ports from 20 to 100 were scanned.
- The scanning method used was a SYN scan.
- Wireshark was used to capture the traffic generated using the loopback capture interface.
- The file name for this capture was **port_scan_lab.pcapng**.
- Wireshark filters were applied on the SYN packets and RST/ACK responses.

4.4.1 SYN Scan Detection

- A Wireshark filter was used to capture packets that had the TCP flag SYN.
- There were many SYN packets captured from one source but headed towards various destination ports.
- There were also many SYN packets captured within a very short period of time.
- That is the kind of behavior that is usually observed during a SYN scan.
- As the scan was conducted on **127.0.0.1**, it was a security test.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	48	41156 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
3	0.000133500	127.0.0.1	127.0.0.1	TCP	48	41156 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.000188700	127.0.0.1	127.0.0.1	TCP	48	41156 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
7	0.000232700	127.0.0.1	127.0.0.1	TCP	48	41156 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
9	0.000275600	127.0.0.1	127.0.0.1	TCP	48	41156 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	0.000316300	127.0.0.1	127.0.0.1	TCP	48	41156 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	0.000362800	127.0.0.1	127.0.0.1	TCP	48	41156 → 54 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
15	0.000411400	127.0.0.1	127.0.0.1	TCP	48	41156 → 56 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
17	0.000457000	127.0.0.1	127.0.0.1	TCP	48	41156 → 60 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
19	0.000499800	127.0.0.1	127.0.0.1	TCP	48	41156 → 55 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
21	0.001941700	127.0.0.1	127.0.0.1	TCP	48	41156 → 72 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
23	0.002015600	127.0.0.1	127.0.0.1	TCP	48	41156 → 30 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
25	0.002139700	127.0.0.1	127.0.0.1	TCP	48	41156 → 73 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
27	0.002181600	127.0.0.1	127.0.0.1	TCP	48	41156 → 59 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
29	0.002228000	127.0.0.1	127.0.0.1	TCP	48	41156 → 87 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	0.002271600	127.0.0.1	127.0.0.1	TCP	48	41156 → 94 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
33	0.002319900	127.0.0.1	127.0.0.1	TCP	48	41156 → 61 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	0.002365300	127.0.0.1	127.0.0.1	TCP	48	41156 → 40 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Frame 1: Packet, 48 bytes on wire (384 bits), 48 bytes captured (384 bits) on interface \Device\NPF_{...} ... 0000 02 00 00 00 45 00 00 2c 43 85 00 00 38 06 41 45 ... E...C...8 AE
Null/Loopback ... 0010 7f 00 00 01 7f 00 00 01 a0 c4 00 17 5a ca e4 40Z...@
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 ... 0020 00 00 00 00 60 02 04 00 b6 3d 00 00 02 04 05 b4=.....
Transmission Control Protocol, Src Port: 41156, Dst Port: 23, Seq: 0, Len: 0

Figure 5: SYN Scan Detection

4.4.2 RST/ACK Responses

- There was a filter on the TCP RST/ACK packets.
- Several RST/ACK replies were recorded.
- RST/ACK replies were sent for quite a number of the probed ports.
- If a TCP RST response is received for a SYN probe, it generally indicates that the port is closed.
- SYN probes along with RST/ACK replies assisted in determining port scanning activity.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000119000	127.0.0.1	127.0.0.1	TCP	44	23 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
4	0.000174400	127.0.0.1	127.0.0.1	TCP	44	21 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
6	0.000219800	127.0.0.1	127.0.0.1	TCP	44	53 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8	0.000263400	127.0.0.1	127.0.0.1	TCP	44	25 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	0.000303900	127.0.0.1	127.0.0.1	TCP	44	22 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12	0.000344200	127.0.0.1	127.0.0.1	TCP	44	80 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14	0.000395200	127.0.0.1	127.0.0.1	TCP	44	54 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
16	0.000441400	127.0.0.1	127.0.0.1	TCP	44	56 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
18	0.000487300	127.0.0.1	127.0.0.1	TCP	44	60 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
20	0.000528200	127.0.0.1	127.0.0.1	TCP	44	55 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
22	0.002000800	127.0.0.1	127.0.0.1	TCP	44	72 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
24	0.002126200	127.0.0.1	127.0.0.1	TCP	44	30 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
26	0.002168800	127.0.0.1	127.0.0.1	TCP	44	73 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
28	0.002211400	127.0.0.1	127.0.0.1	TCP	44	59 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
30	0.002258900	127.0.0.1	127.0.0.1	TCP	44	87 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
32	0.002302900	127.0.0.1	127.0.0.1	TCP	44	94 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
34	0.002350000	127.0.0.1	127.0.0.1	TCP	44	61 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
36	0.002396900	127.0.0.1	127.0.0.1	TCP	44	49 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
38	0.002443800	127.0.0.1	127.0.0.1	TCP	44	75 → 41156 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Frame 2: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_{...} ... 0000 02 00 00 00 45 00 00 28 52 08 40 00 80 06 aa c5 ... E...R...@...
Null/Loopback ... 0010 7f 00 00 01 7f 00 00 01 00 17 a0 c4 00 00 00 00Z...AP...
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 ... 0020 5a ca e4 41 50 14 00 00 d1 e6 00 00=.....
Transmission Control Protocol, Src Port: 23, Dst Port: 41156, Seq: 1, Ack: 1, Len: 0

Figure 6: RST/ACK Responses

5. NETWORK TRAFFIC ANALYSIS

5.1 DNS Query and Response Analysis

- The **dns** display filter was applied.
- DNS communication between local system and DNS resolver was analyzed.
- Analysis of query for **www.youtube.com** was performed.
- A query for **A record** was done.
- The response contained **several A records**.
- There was **No error** in response.
- The measured response time was **4.75 ms**.
- This showed the normal DNS resolution process.

Refer to the DNS query and response packets shown in **Figure 1** and **Figure 2** .

5.2 TCP Three-Way Handshake Analysis

- SYN Packet started the connection.
- SYN/ACK confirmed it.
- ACK packet completed the handshake.
- The above proves the normal TCP connection establishment procedure.

The TCP handshake packets are shown in **Figure 3**.

5.3 TLS Client Hello Analysis

- Identification of TLS 1.3 Client Hello took place.
- SNI extension was observed.
- SNI showed www.youtube.com.
- TLS protocol was used to establish encrypted communication.

The TLS Client Hello packet is shown in **Figure 4**.

6. SUSPICIOUS ACTIVITY DETECTION

6.1 SYN Port Scan Analysis

- A SYN scan using Nmap was conducted.
- The scan was done on the local machine 127.0.0.1.
- Ports between 20 and 100 were scanned.
- Multiple SYN packets were captured by Wireshark.
- It was evident that the same source was targeting different destination ports.
- These packets were within a very short time frame.
- Such a scenario depicts port scanning.
- This was a controlled security experiment since the target was the local machine.

The SYN scan activity observed in the captured traffic is shown in **Figure 5**.

6.2 RST/ACK Response Analysis

- Wireshark was filtered for the RST/ACK packets.
- Several RST/ACK replies were detected.
- They matched quite a number of the ports that were scanned.
- A TCP RST response to a SYN probe generally indicates that the destination port is closed.
- Both the RST/ACK replies and the SYN probes indicated the presence of a controlled port scan.

The RST/ACK responses observed during the scan are shown in **Figure 6**.

7. PROTOCOL HIERARCHY ANALYSIS

7.1 Protocol Distribution

- The Protocol Hierarchy feature of Wireshark was used in the analysis of the protocols in the baseline capture.
- There was both IPv4 and IPv6 traffic in the baseline capture.
- There was TCP and UDP traffic in the baseline capture.
- QUIC over UDP made up a considerable number of the packets in the baseline capture.
- There was TLS traffic in the baseline capture.
- DNS traffic over IPv4 and IPv6 was also seen in the baseline capture.

The protocol distribution observed in the baseline capture is shown in **Figure 7**.

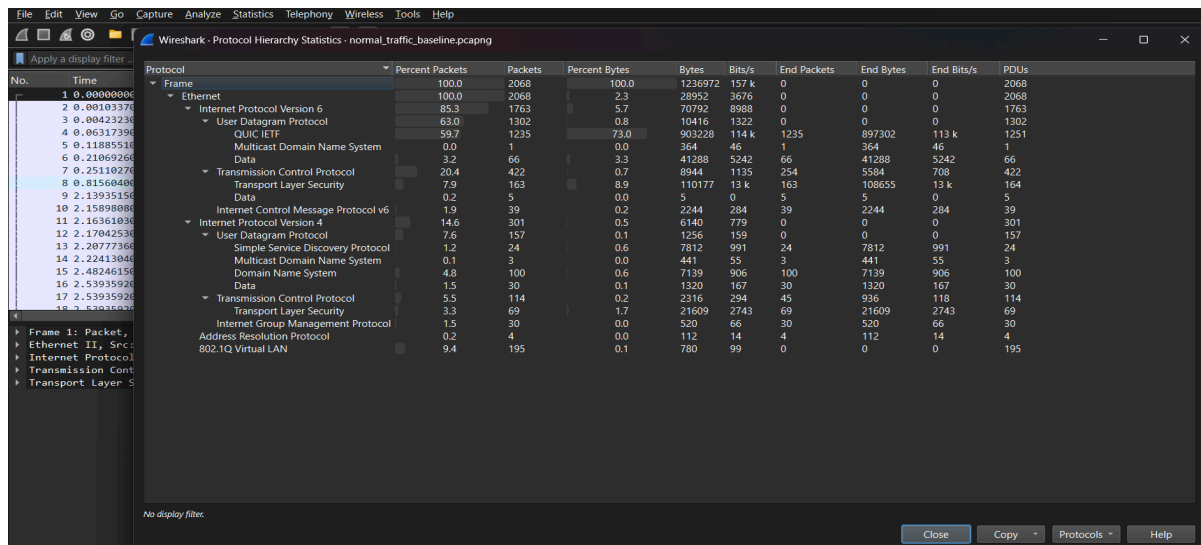


Figure 7: Protocol Hierarchy

8. KEY FINDINGS

Analysis	Observation	Security Relevance
DNS Analysis	DNS queries and responses were observed during normal browsing.	Helps understand domain resolution and identify unusual DNS activity.
TCP Analysis	A TCP three-way handshake was observed.	Helps identify normal connection establishment and unusual connection attempts.
TLS Analysis	TLS 1.3 Client Hello and SNI information were observed.	Shows how encrypted connections can still expose some connection metadata.
Protocol Analysis	TCP, UDP, QUIC, DNS, TLS, IPv4 and IPv6 traffic were observed.	Provides an overview of normal network traffic patterns.
SYN Port Scan	Multiple SYN packets were sent to different ports on the local host.	A repeated pattern of this type can indicate network reconnaissance.
RST/ACK Responses	RST/ACK responses were observed for many probed ports.	Helps identify closed ports during a port scan.

9. CONCLUSION

By engaging in the project, I was able to gain hands-on experience in analyzing network traffic using the Wireshark tool, as well as performing a SYN port scan using Nmap. Network traffic capturing was useful in understanding DNS queries and responses, the establishment of TCP connections, the TLS Client Hello message, and the protocols used in the network.

Controlled SYN port scanning has proven to be important in understanding how SYN packets can be sent to different ports and how RST/ACK packets can be used to establish which ports are closed. This has been important in understanding how network traffic and packets can be used to detect suspicious network activity.

Overall, the project has been important in gaining practical experience in network traffic analysis and network reconnaissance.

10. REFERENCES

1. Wireshark Foundation. *Wireshark User's Guide* [wireshark.org](https://www.wireshark.org).
2. Nmap Project. *Nmap Reference Guide* nmap.org.
3. Nmap Project. *Nmap Documentation* nmap.org.
4. Npcap. *Npcap Documentation*.